

IMAGO Media Search — Submission

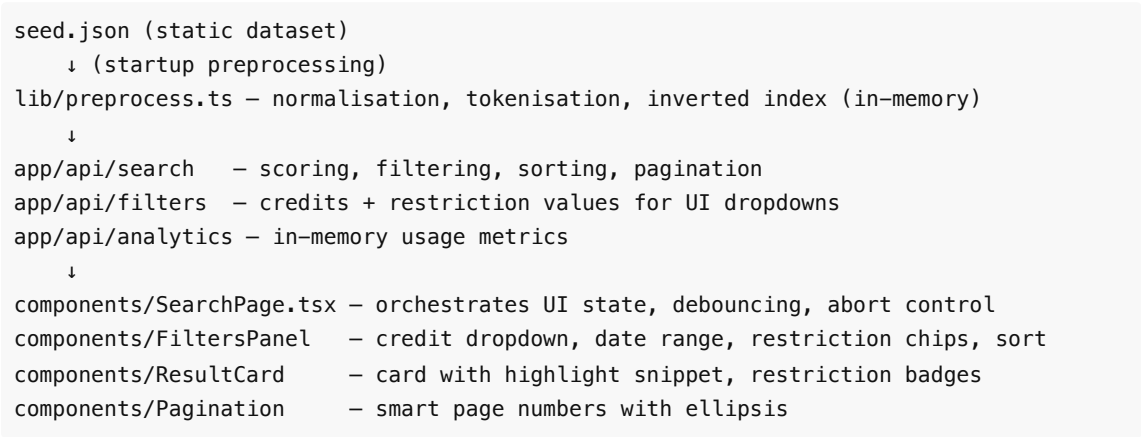
Live demo: <https://imago-search-teal.vercel.app>

Repository: <https://github.com/v-arui/IMAGO-Search>

Stack: Next.js 16 (App Router) · TypeScript · Tailwind CSS

1. Architecture Overview

The application is structured in three layers:



Key architectural decision: everything runs in a single Next.js server process. The inverted index lives as a module-level singleton, built once on first request and reused for the lifetime of the process. This avoids the complexity of an external search service while keeping request latency in the 4–15ms range for 10,000 items.

2. Search Strategy & Relevance

Tokenisation

Each `suchtext` is processed through a normalisation pipeline:

1. Lowercase
2. Strip punctuation (preserve German umlauts ä/ö/ü/ß and intra-word hyphens)
3. Split on whitespace
4. Remove tokens shorter than 2 characters
5. Remove stop words (bilingual German/English set)

The same pipeline runs on user queries at search time, ensuring consistent matching regardless of casing or punctuation.

Inverted Index

A `Map<token, Map<itemId, score>>` is built at startup. This gives $O(1)$ token lookup rather than $O(n)$ full-scan per query.

Scoring & Field Weights

Field	Weight	Rationale
-------	--------	-----------

suchtext	1.0	Primary content — most semantically rich
fotografen	0.5	Agency/credit search is secondary intent
bildnummer	0.3	Identifier lookup — useful but not a relevance signal

Scores accumulate across tokens. A document matching two query terms outranks one matching only one.

Prefix Matching

For each query token, two passes are made:

1. **Exact match** — full score from posting list
2. **Prefix match** — 60% weight, so typing "Muell" surfaces "Mueller" before the full word is typed

Prefix matching is $O(V)$ over the vocabulary, which is acceptable for 10k items. A trie or dedicated search engine would handle this more efficiently at scale.

Relevance Sort Tiebreaker

When two documents score equally, the more recent one (`datumIso` descending) is ranked higher. This biases results toward current content, which suits a media library.

3. Preprocessing Strategy

All preprocessing runs **once at server startup** (lazy, on first request), not per-request. The results are cached in module-level singletons.

What is preprocessed

Date normalisation (`DD.MM.YYYY` → `YYYY-MM-DD`)

The seed data stores dates in German format. Converting to ISO 8601 makes dates lexicographically sortable and directly comparable with the `YYYY-MM-DD` strings produced by HTML date inputs — no Date object parsing needed at query time.

Restriction extraction (regex over `suchtext`)

Publication restriction tokens (`PUBLICATIONxINxGERxONLY` , etc.) are embedded in the free-text `suchtext` field with no separate structured field. A regex (`/PUBLICATION[A-Za-z]+(?:x[A-Za-z]+)*x/g`) extracts them into a dedicated `restrictions[]` array per item. The filter path then uses a simple `Array.includes()` — no string parsing at query time.

Tokenisation and stop-word removal

Tokens are computed once per item and stored. At query time, only the (short) user query is tokenised — not the 10,000 `suchtexts`.

Inverted index construction

Token → posting list map built from all preprocessed tokens. Converts $O(n)$ full-scan search into $O(k)$ posting list lookup where k = number of matching documents.

Why build-time vs runtime

Preprocessing at startup amortises the cost (~60ms for 10k items) across all requests. Per-request preprocessing would add 60ms to every search, dominating the measured 4–15ms query latency.

4. Scaling to Millions of Items

The current in-memory approach reaches its limits at around 100k items due to memory pressure and startup latency. The path to scale:

Search engine

Replace the in-memory index with **Meilisearch** (simpler, self-hosted) or **Elasticsearch** (more mature, cloud-hosted). Both handle:

- Inverted indexing and BM25 relevance scoring natively
- Prefix/fuzzy matching at $O(\log V)$
- Faceted filtering without post-filtering the full result set
- Horizontal read scaling via replica nodes

The preprocessing pipeline (date normalisation, restriction extraction) would run at **ingestion time**, writing structured fields (`datumIso` , `restrictions[]`) alongside the raw text into the search engine index.

Continuous ingestion

```
New/updated media item
  → Ingestion queue (Kafka / SQS)
  → Preprocessing worker (normalise, extract, tokenise)
  → Upsert into search engine index
  → Cache invalidation for affected filter values
```

This decouples indexing latency from query latency. New items appear in search results within seconds of ingestion without blocking the API.

Caching

- Popular queries: Redis or CDN-level caching (e.g. Vercel Edge Cache) with short TTLs
- Filter values (`/api/filters`): cache aggressively — changes rarely
- SEO pages: ISR (Incremental Static Regeneration) for common query URLs to serve organic traffic without hitting the search engine on every request

SEO / Organic discoverability

For a media platform, organic traffic is significant. At scale:

- Generate static pages for the most-searched terms (`/search/chelsea-fc` , `/search/taylor-swift`)
- Add structured data (`JSON-LD ImageObject`) to result pages
- Build a sitemap from top search terms and popular bildnummer values
- Use Next.js ISR to keep these pages fresh without full rebuilds

5. Testing Approach

Given the time constraint, tests were not implemented. In a production codebase, the priority order would be:

Unit tests (highest ROI)

- `tokenize()` — edge cases: empty string, umlauts, hyphens, stop words, single chars
- `parseDatum()` — valid dates, malformed input, edge dates (`01.01.1900`)
- `extractRestrictions()` — known tokens, multiple tokens per suchtext, no tokens
- Scoring logic — verify field weights, prefix discount, multi-token accumulation

Integration tests

- `GET /api/search` — keyword returns correct items; filter by credit excludes others; date range boundary conditions; page/pageSize math; invalid params return sensible results (not 500)
- `GET /api/filters` — returns sorted, deduplicated credits and restrictions

End-to-end

- Search flow: type query → debounce fires → results appear → filter narrows results → clear resets
- Pagination: navigate to page 3, change filter, assert reset to page 1
- Error state: mock API failure, assert error banner appears with retry

Tools

Vitest for unit/integration (fast, native ESM), Playwright for E2E.

6. Trade-offs & Scope Decisions

Decision	What was built	What was deferred	Why
Search engine	In-memory inverted index	Meilisearch / Elasticsearch	Zero infrastructure, self-contained demo
Prefix matching	O(V) vocabulary scan	Trie / engine-native prefix	Fast enough at 10k, understandable code
Stop words	Minimal ~50-word bilingual list	Full NLP tokenizer with language detection	Sufficient for the corpus, no dependency
Analytics	In-memory, resets on restart	Persisted to database	Meets demo requirement, no infra needed
Images	Aspect-ratio placeholder	Real thumbnails	No image URLs in seed data
Tests	None	Vitest + Playwright suite	Time constraint; testing approach documented
Deployment	Vercel (serverless)	Containerised Meilisearch	Simplest path to a live URL
PDF	This document	Interactive architecture diagram	Content-equivalent, faster to produce
Multi-language stop words	German + English	Full language detection	Corpus is mixed but manageable with a small list

Query mode	OR semantics (any token)	AND / phrase mode	More forgiving for exploratory search; power users can use exact bildnummer
------------	-----------------------------	-------------------	---

Running Locally

```
git clone https://github.com/v-arv/IMAGO-Search
cd IMAGO-Search
npm install
npm run dev
```

Open <http://localhost:3000>.